Academic Question Answering System using RAG

Project Documentation — Version 1

Retrieval-Augmented Generation for Academic Notes

1. TITLE

Academic Question Answering System using Retrieval-Augmented Generation (RAG). Version 1 is a cloud-based prototype that accepts academic PDF notes, converts the document into searchable vector representations, retrieves relevant passages for a student's question, and generates a grounded answer using an external large language model.

2. ABSTRACT

This project implements a Retrieval-Augmented Generation (RAG) based academic question-answering system. The current V1 prototype is tested with a six-page concise Operating Systems notes PDF. The document is loaded with LangChain's PyPDFLoader and divided into smaller overlapping text chunks using RecursiveCharacterTextSplitter. Each chunk is converted into a 1024-dimensional embedding using Jina's hosted jina-embeddings-v5-text-small model. The embeddings and associated metadata are stored in a Pinecone serverless vector index using cosine similarity.

At query time, the user's question is embedded using the same Jina model. Pinecone retrieves the most semantically relevant chunks. V1 includes adaptive retrieval: ordinary questions use a smaller top-k value, conceptual questions use a larger context, and broad questions such as complete-topic requests can retrieve substantially more chunks. The retrieved text is then supplied as context to a Gemini model through the Google GenAI SDK. The generation prompt explicitly instructs the model to answer only from the supplied context and to state when the answer cannot be found.

The current prototype demonstrates the complete cloud-based pipeline from PDF ingestion to grounded answer generation through a Streamlit-based user interface. The system provides an interactive chat interface for submitting academic questions, displaying generated answers, and viewing retrieved source chunks. It is intentionally designed as v1 so that subject-aware filtering, a production grade user interface, larger academic datasets, evaluation metrics, and production deployment can be added in later versions.

3. TECH STACK

The V1 implementation uses the following technologies and services:

Layer	Technology	Purpose
Language / Runtime	Python + Jupyter Notebook/Streamlit	Implementation and experimentation
PDF Processing	PyPDF / LangChain PyPDFLoader	Extract text from academic PDF files
Chunking	RecursiveCharacterTextSplitter	Split extracted text into retrieval-sized chunks
Embeddings	Jina AI — jina-embeddings-v5-text-small	Convert document/query text into vectors
Vector Database	Pinecone	Store and similarity-search document vectors
Similarity	Cosine similarity	Rank semantically similar chunks
LLM	Google Gemini via google-genai	Generate the final grounded answer
HTTP / API	Requests	Call Jina's hosted embedding API
Configuration	python-dotenv	Load API keys from environment variables
User Interface	Streamlit	Provide the interactive web-based chat interface for academic questions, generated answers, conversation history, and retrieved sources

API Configuration

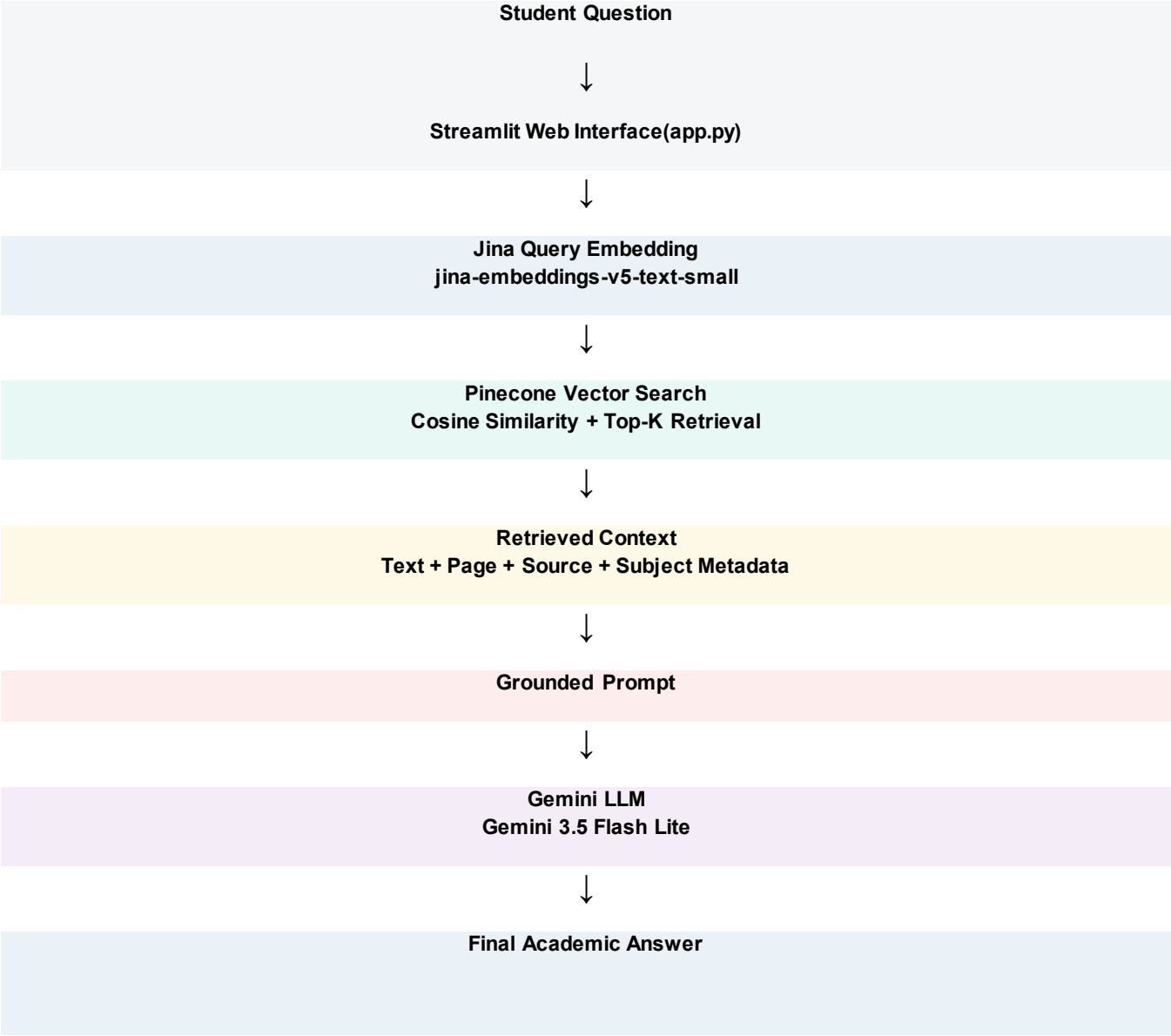
API credentials are loaded through environment variables rather than being hard-coded in the notebook. The current configuration uses JINA_API_KEY, PINECONE_API_KEY, and GEMINI_API_KEY.

Key V1 Parameters

- Jina embedding model: jina-embeddings-v5-text-small.
- Observed embedding dimension in the notebook: 1024.
- Pinecone metric: cosine similarity.
- Jina document embedding batch size: 50 chunks.
- Pinecone upload batch size: 100 records.
- OS document: 6-page concise notes PDF.
- Observed OS document chunk count: 26.
- OS vector IDs use the os-chunk-{i} pattern to avoid collisions with earlier records.

4. FLOW + ARCHITECTURE

V1 RAG Architecture



4.1 Document Ingestion Flow

Academic PDF

↓

PyPDFLoader

↓

Extracted page documents

↓

RecursiveCharacterTextSplitter

↓

Text chunks

↓

Jina Embedding API

↓

1024-D vectors

↓

Pinecone serverless index

4.2 Query / RAG Flow

Student question

↓

Jina query embedding

↓

Pinecone vector similarity search

↓

Retrieved chunks + page metadata

↓

Grounded prompt

↓

Gemini generation

↓

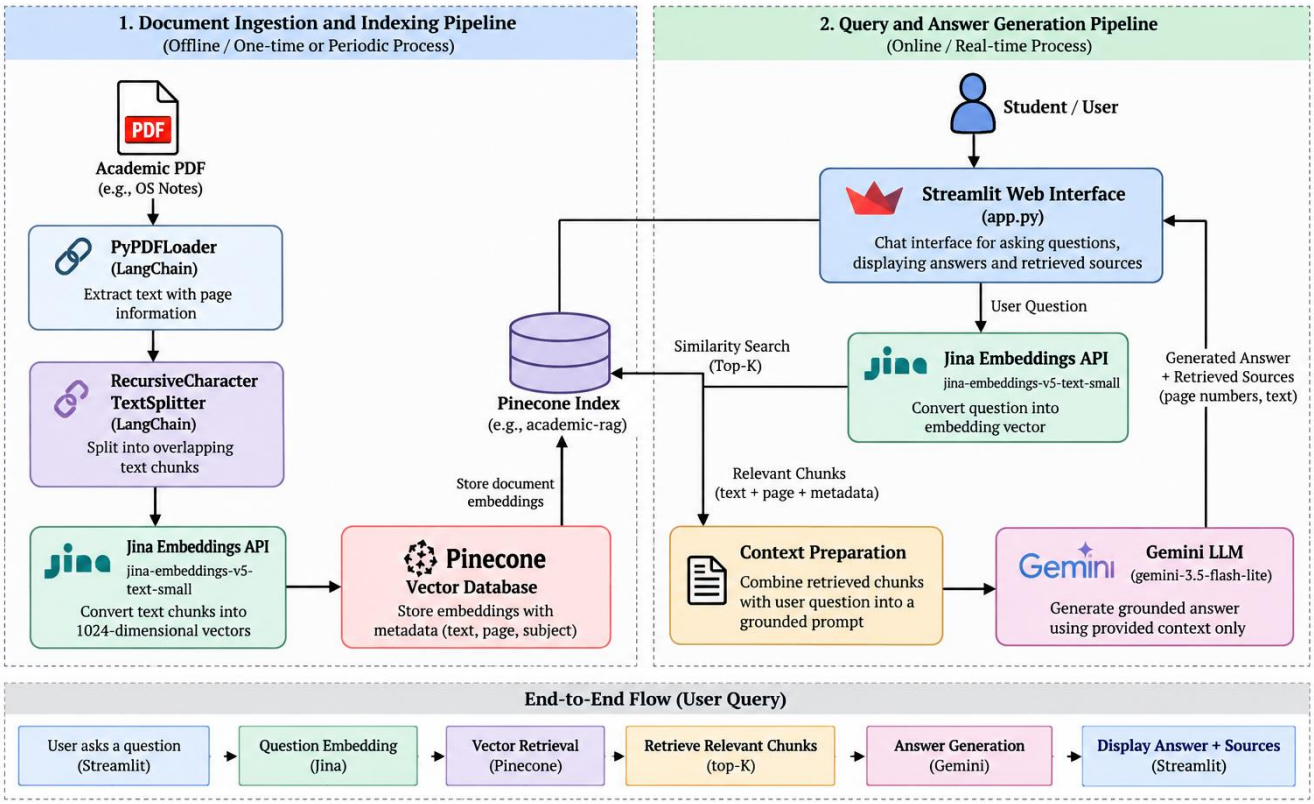
Final academic answer

4.3 Architecture Explanation

The Streamlit application provides the user-facing interface through which questions are submitted and generated answers and retrieved sources are displayed. The architecture separates retrieval from generation. Jina is responsible for semantic representation, Pinecone is responsible for storing and retrieving relevant vectors, and Gemini is responsible for natural-language answer generation. This reduces the need to place the entire PDF into the LLM prompt and provides a mechanism for grounding answers in the indexed material.

Each Pinecone record contains the chunk text, source filename, page number, subject label, vector, and a unique identifier. In V1, the subject metadata is stored for the Operating Systems document; subject-aware filtering is planned for the multi-subject version.

Figure 3. System Architecture Diagram
Academic RAG System (V1)



5. DIRECTORY STRUCTURE

V1 currently contains only four project files: app.py, os_notes.pdf, rag.ipynb, and .env. The notebook contains the complete RAG pipeline and is intentionally kept as the main implementation artifact for V1.

V1 contains the document-processing notebook, Streamlit application, academic source document, and environment configuration. The notebook is used for document ingestion, embedding generation, Pinecone indexing, experimentation, and testing, while `app.py` provides the user-facing Streamlit application. A practical project structure for preserving the current workflow and preparing it for later expansion is shown below.

```
Academic-RAG/

├── app.py
├── os_notes.pdf
├── rag_v1.ipynb
└── .env
```

Current Notebook Organization

- Project Configuration — package installation, imports, and environment variables.
- PDF Ingestion & Text Chunking — loads the Operating Systems PDF and creates overlapping chunks.
- Jina Embeddings — generates document vectors in batches using jina-embeddings-v5-text-small.
- Pinecone Vector Database — creates/connects to the academic-rag index, prepares metadata, uploads vectors, and verifies statistics.
- Query Embedding & Retrieval — embeds questions and retrieves relevant chunks from Pinecone.
- Adaptive Top-K Retrieval — selects the retrieval depth based on the type of question.
- Gemini Answer Generation — constructs grounded context and generates the final answer.
- End-to-End RAG Test — accepts a user question and runs the complete pipeline.
- Optional Testing — provides suggested questions for functional testing and retrieval inspection.
- Retrieval Part: installs the PDF/chunking dependencies and loads the source document.
- Embedding Stage: loads credentials and sends text batches to Jina.
- Pinecone Stage: creates/opens the academic-rag index, prepares metadata records, uploads vectors, and checks index statistics.
- LLM Generation Stage: initializes Gemini and defines the query embedding, retrieval, adaptive top-k, and answer-generation functions.
- Final Test Stage: accepts a question, retrieves relevant context, and prints the generated answer.

Metadata Record

The V1 OS records store text, page number, source filename, and subject. This structure is important for future multi-subject retrieval because the same Pinecone index can contain documents from different courses.

6. TESTING + PERFORMANCE

6.1 Functional Testing

The pipeline was executed successfully with the six-page Operating Systems concise notes PDF. The extraction stage produced 26 chunks. All 26 chunks were embedded successfully and uploaded to Pinecone.

Test	Observed Result	Status
PDF loading	6-page OS PDF loaded successfully	PASS
Chunking	26 chunks generated	PASS
Jina embeddings	26 vectors; 1024 dimensions	PASS
Pinecone connection	academic-rag index connected	PASS
Vector upload	26/26 OS records uploaded	PASS
Vector statistics	Pinecone reported 257 total vectors in the existing index	PASS
Question retrieval	Relevant OS chunks retrieved	PASS
Answer generation	Grounded answer generated by Gemini	PASS

6.2 Retrieval Strategy

V1 uses adaptive top-k retrieval. The retrieval function can use a smaller number of chunks for focused questions and a larger number for conceptual or broad questions. Broad keywords such as complete, entire, all topics, full syllabus, and overview are mapped to a larger retrieval window. This is intended to reduce the risk of missing information that is distributed across multiple chunks.

6.3 Performance Observations

- Embedding throughput is batched at 50 text chunks per Jina API request.
- Pinecone uploads are batched at 100 records per upsert operation.
- The observed embedding output is 1024-dimensional, providing a fixed vector size for the current index.
- The current notebook does not record formal end-to-end latency, precision/recall, MRR, or answer-faithfulness scores; therefore no numerical accuracy or latency claim is made in V1.
- Functional testing showed that the OS notes could be retrieved and used to answer questions such as paging-related questions from the indexed context.

6.4 Current V1 Limitations and Next Version

- V1 uses a Jupyter/VS Code-based document processing and indexing workflow with a Streamlit-based web interface. The application is currently a local prototype and is not deployed as a production-grade service.
- The current test focuses on one Operating Systems PDF.
- Subject metadata is stored but is not yet used as a Pinecone query filter.
- A future multi-subject version can add DBMS, Web Technology, DAA, and other academic documents to the same vector database.
- Future evaluation should measure retrieval recall, answer faithfulness, response latency, and question-level accuracy using a labeled test set.
- Future versions can extend the existing Streamlit interface with multi-user authentication, subject selection, advanced filtering, evaluation dashboards, API access, and production cloud deployment.

End of Project V1 Documentation